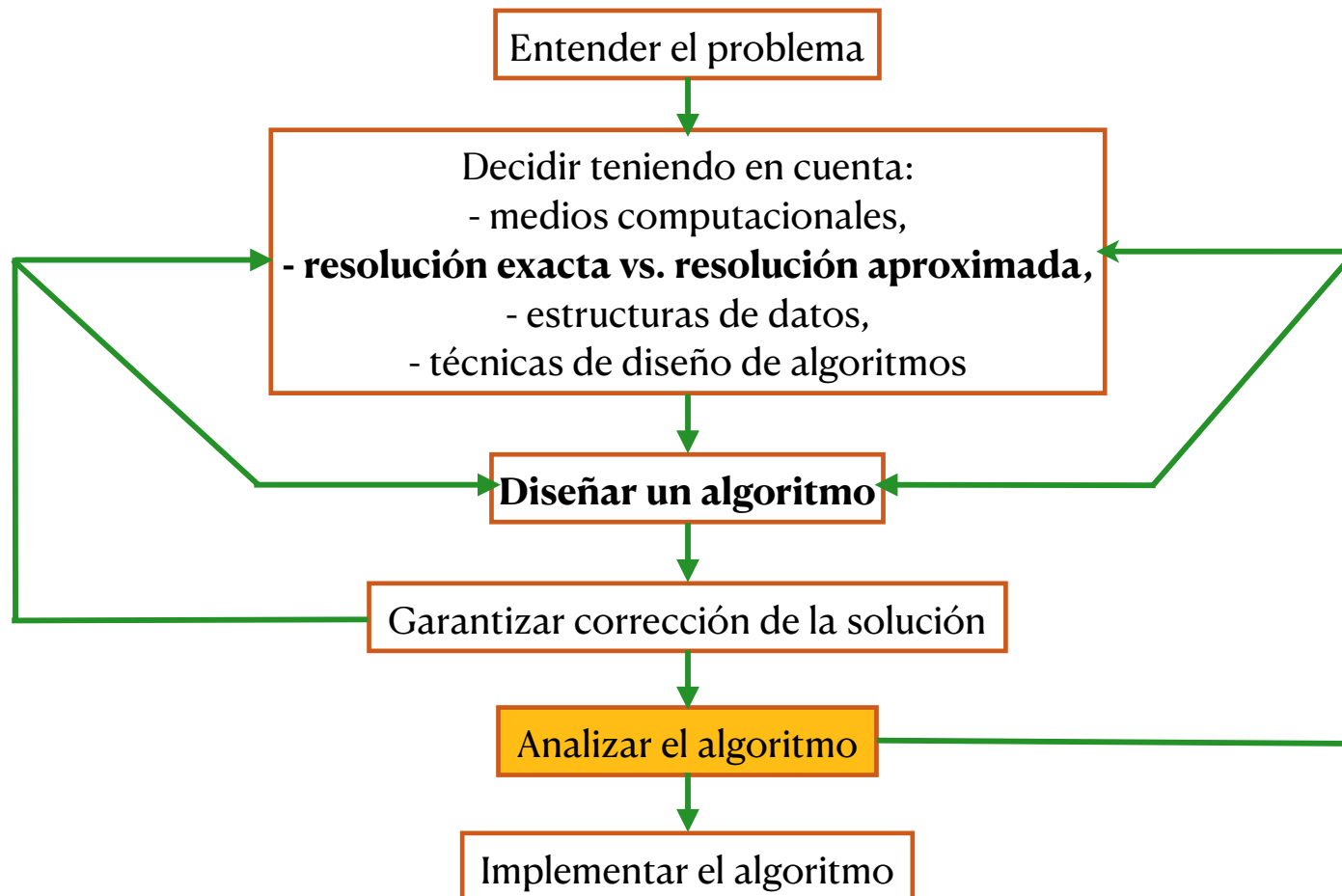


Análisis y Diseño de Algoritmos I

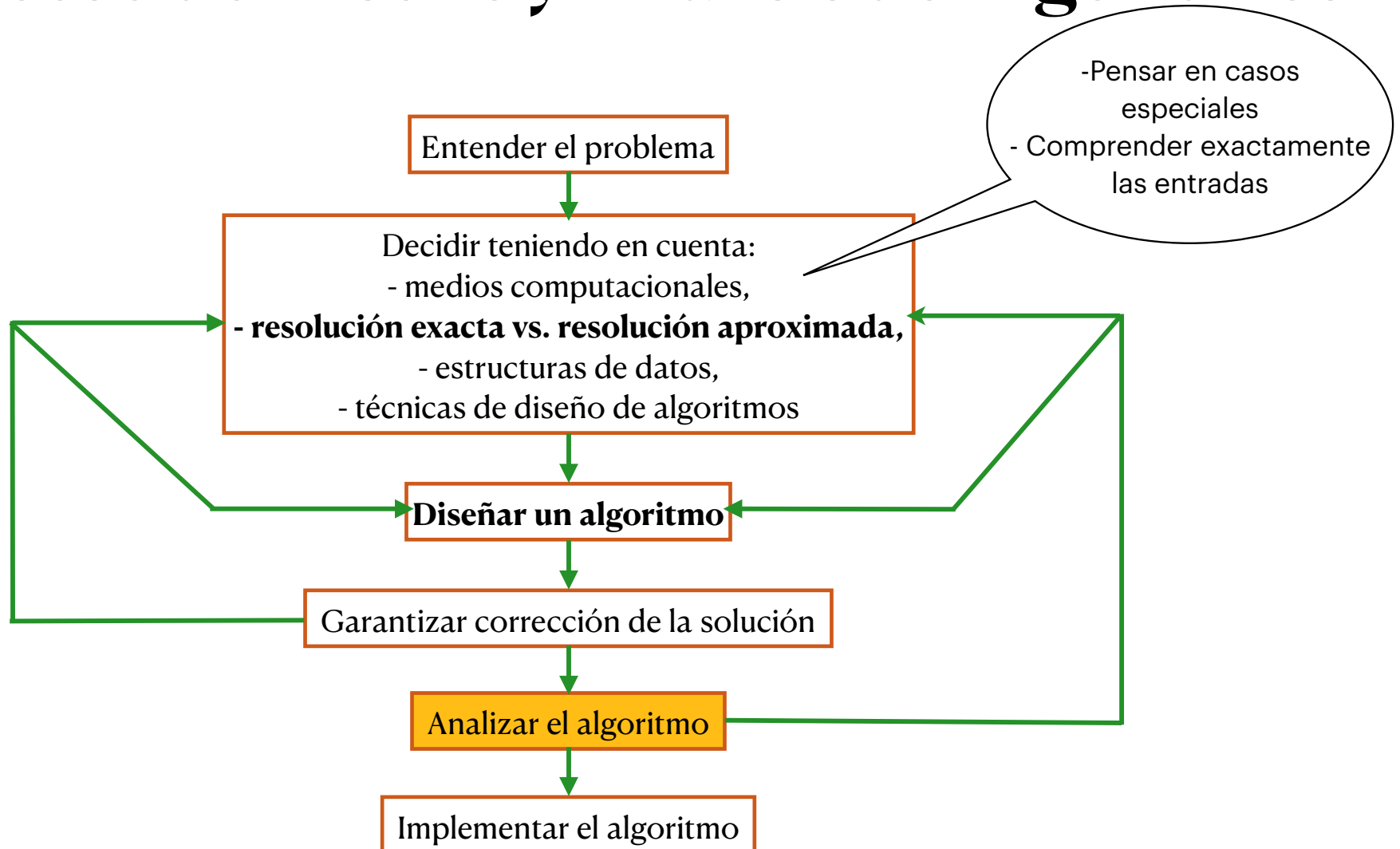
Valeria Bengolea
Departamento de Computación
Facultad de Ciencias Exactas, Físico-Químicas y Naturales
Universidad Nacional de Río Cuarto

Análisis de Algoritmos

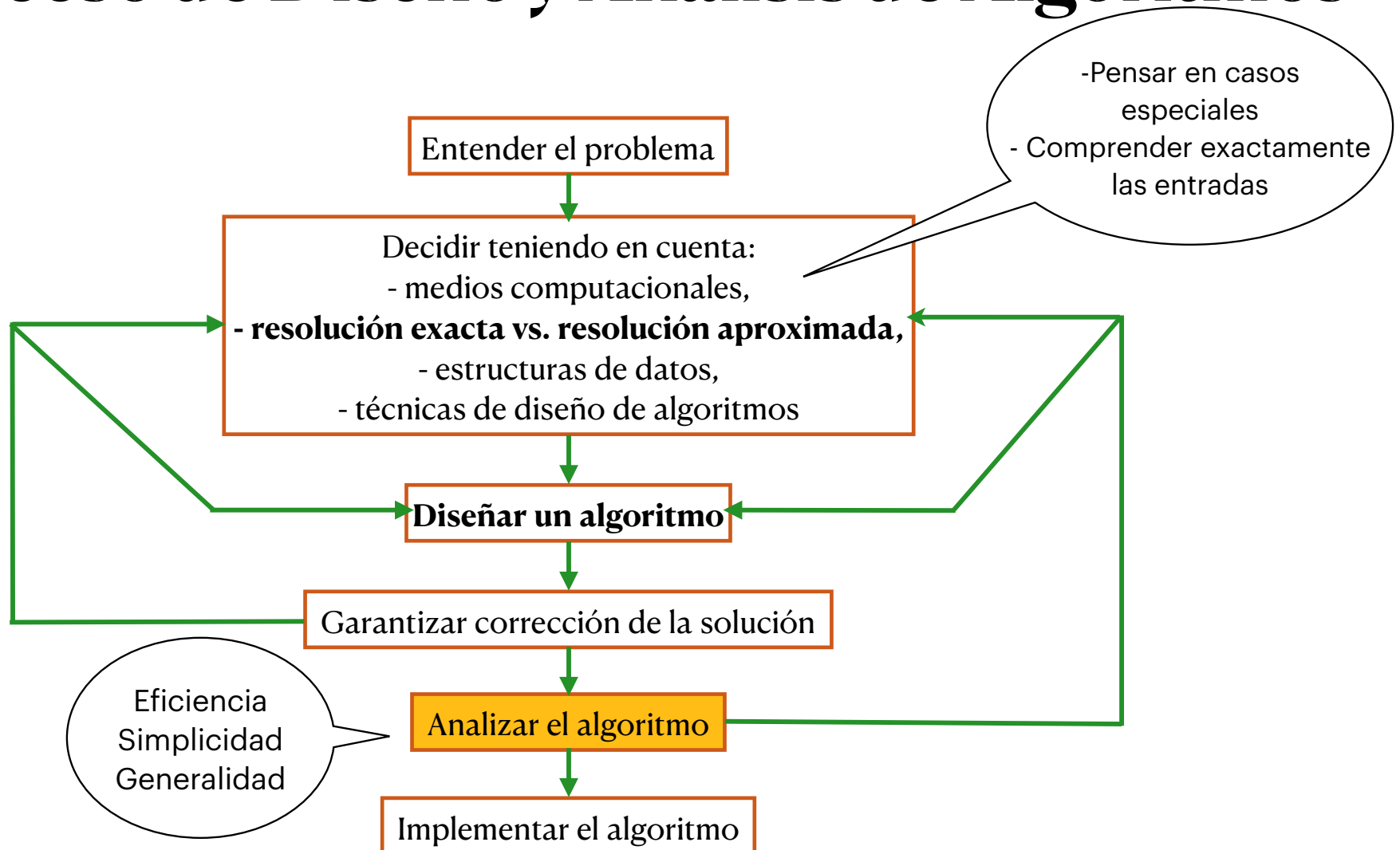
El Proceso de Diseño y Análisis de Algoritmos



El Proceso de Diseño y Análisis de Algoritmos



El Proceso de Diseño y Análisis de Algoritmos



Análisis de Algoritmos

En la construcción de algoritmos es necesario realizar tareas de análisis para:

- corrección
- eficiencia en tiempo
- eficiencia en espacio ocupado

...

Enfoques

- analítico
- empírico

Análisis de Eficiencia en Tiempo

El análisis de eficiencia en tiempo se realiza determinando el número de veces que la operación de interés es ejecutada, como una función sobre el tamaño de la entrada.

La operación de interés es aquella que contribuye en mayor medida al tiempo de ejecución del algoritmo estudiado.

Análisis de Eficiencia en Tiempo

El análisis de eficiencia en tiempo se realiza determinando el número de veces que la operación de interés es ejecutada, como una función sobre el tamaño de la entrada.

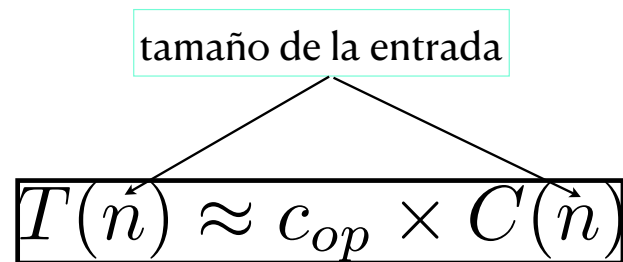
La operación de interés es aquella que contribuye en mayor medida al tiempo de ejecución del algoritmo estudiado.

$$T(n) \approx c_{op} \times C(n)$$

Análisis de Eficiencia en Tiempo

El análisis de eficiencia en tiempo se realiza determinando el número de veces que la operación de interés es ejecutada, como una función sobre el tamaño de la entrada.

La operación de interés es aquella que contribuye en mayor medida al tiempo de ejecución del algoritmo estudiado.



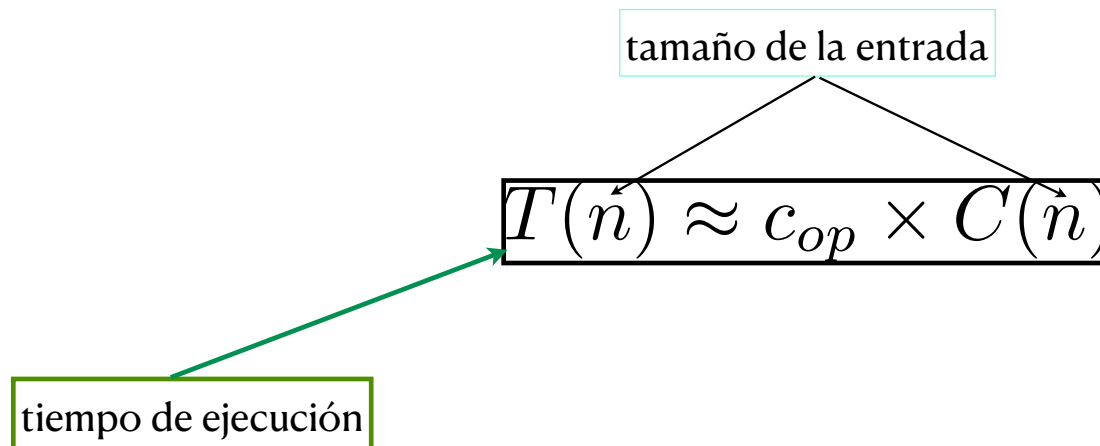
A diagram illustrating the relationship between input size and time complexity. A light blue rectangular box at the top contains the text "tamaño de la entrada". Two black lines originate from the bottom corners of this box and point downwards to the variables $\vec{n}$ in the expression $T(\vec{n}) \approx c_{op} \times C(\vec{n})$. This expression is enclosed in a black rectangular box.

$$T(\vec{n}) \approx c_{op} \times C(\vec{n})$$

Análisis de Eficiencia en Tiempo

El análisis de eficiencia en tiempo se realiza determinando el número de veces que la operación de interés es ejecutada, como una función sobre el tamaño de la entrada.

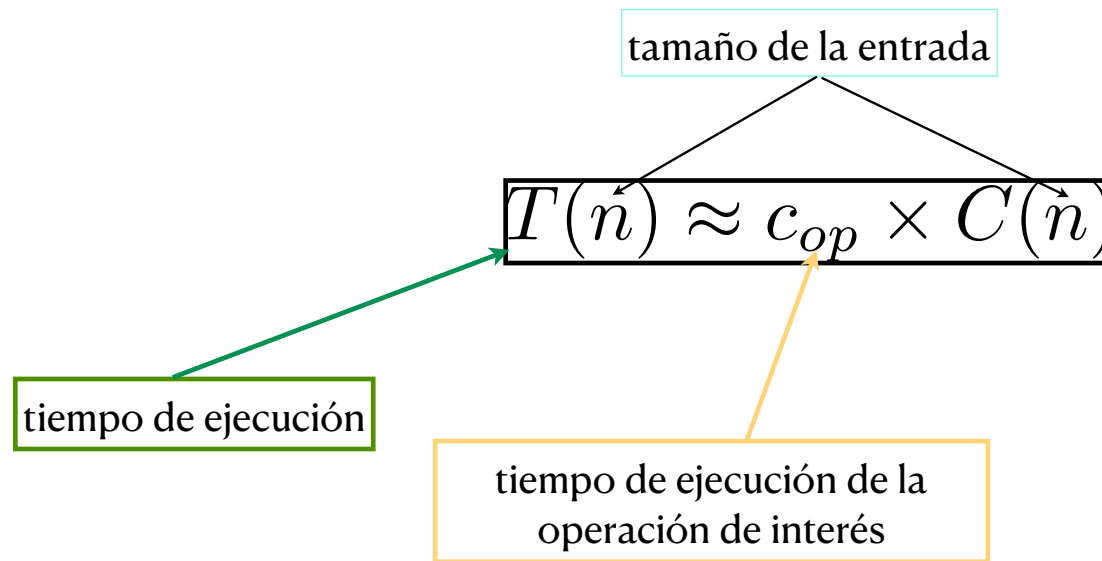
La operación de interés es aquella que contribuye en mayor medida al tiempo de ejecución del algoritmo estudiado.



Análisis de Eficiencia en Tiempo

El análisis de eficiencia en tiempo se realiza determinando el número de veces que la operación de interés es ejecutada, como una función sobre el tamaño de la entrada.

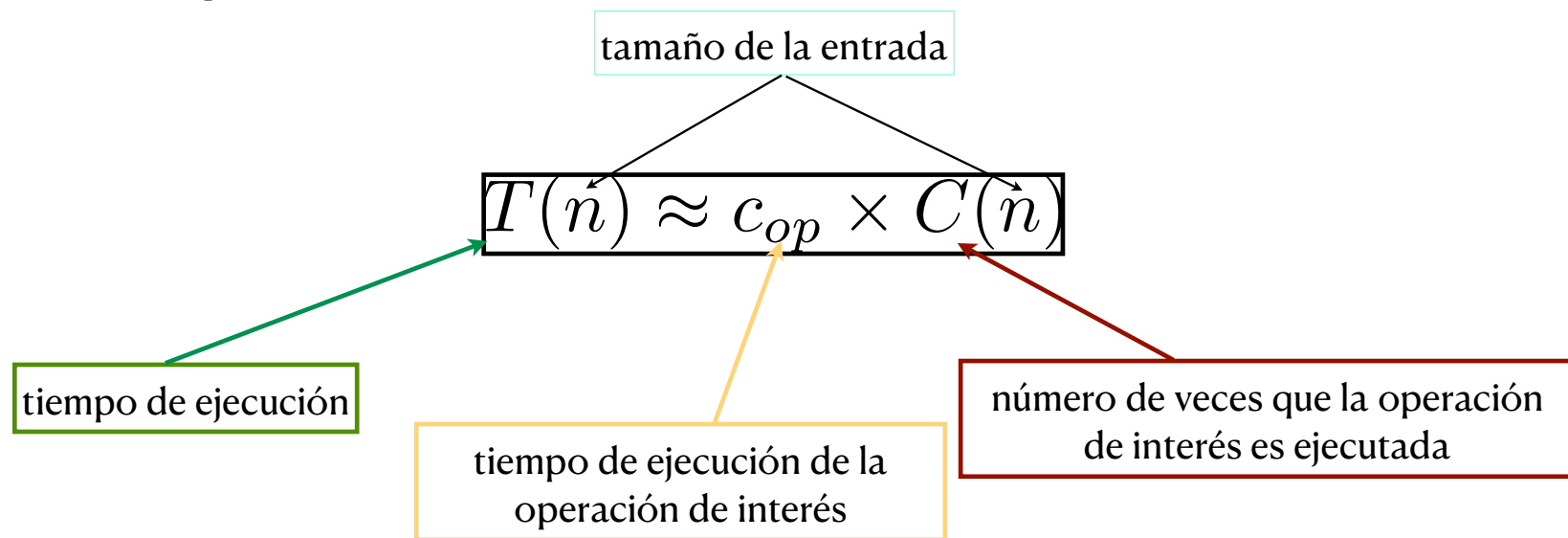
La operación de interés es aquella que contribuye en mayor medida al tiempo de ejecución del algoritmo estudiado.



Análisis de Eficiencia en Tiempo

El análisis de eficiencia en tiempo se realiza determinando el número de veces que la operación de interés es ejecutada, como una función sobre el tamaño de la entrada.

La operación de interés es aquella que contribuye en mayor medida al tiempo de ejecución del algoritmo estudiado.



Ejemplos de Tamaños de Entrada y Operaciones de Interés

Ejemplos de Tamaños de Entrada y Operaciones de Interés

Problema	Medida del tamaño de la entrada	Operación de interés
----------	---------------------------------	----------------------

Ejemplos de Tamaños de Entrada y Operaciones de Interés

Problema	Medida del tamaño de la entrada	Operación de interés
Búsqueda de una clave en una lista de n elementos	Número de elementos de la lista, i.e., n	Comparación de claves

Ejemplos de Tamaños de Entrada y Operaciones de Interés

Problema	Medida del tamaño de la entrada	Operación de interés
Búsqueda de una clave en una lista de n elementos	Número de elementos de la lista, i.e., n	Comparación de claves
Multiplicación de matrices	Dimensiones de las matrices, o el número total de elementos	Multiplicación de números

Ejemplos de Tamaños de Entrada y Operaciones de Interés

Problema	Medida del tamaño de la entrada	Operación de interés
Búsqueda de una clave en una lista de n elementos	Número de elementos de la lista, i.e., n	Comparación de claves
Multiplicación de matrices	Dimensiones de las matrices, o el número total de elementos	Multiplicación de números
Chequeo de primalidad de un entero n	el tamaño de n , i.e., el número de dígitos de n (en representación binaria)	División

Ejemplos de Tamaños de Entrada y Operaciones de Interés

Problema	Medida del tamaño de la entrada	Operación de interés
Búsqueda de una clave en una lista de n elementos	Número de elementos de la lista, i.e., n	Comparación de claves
Multiplicación de matrices	Dimensiones de las matrices, o el número total de elementos	Multiplicación de números
Chequeo de primalidad de un entero n	el tamaño de n , i.e., el número de dígitos de n (en representación binaria)	División
Algún problema típico de grafos	número de vértices y/o número de arcos	Visitar un vértice o atravesar un arco

Peor caso, caso promedio y mejor caso

En muchos casos, la eficiencia de un algoritmo no sólo depende del tamaño de la entrada, sino también de su forma. En estos casos, en el análisis del tiempo de ejecución de un algoritmo puede distinguirse entre:

- análisis de peor caso. : máximo entre las entradas de tamaño n
- análisis de mejor caso. : mínimo entre las entradas de tamaño n
- análisis de caso promedio. : promedio entre las entradas de tamaño n
 - el número de veces que la operación de interés será ejecutada para una entrada “típica”
 - NO es el promedio entre el mejor caso y el peor caso
 - el número esperado de operaciones de interés es interpretado como una variable aleatoria (con alguna suposición de la distribución de las posibles entradas)

Peor caso, caso promedio y mejor caso

En muchos casos, la eficiencia de un algoritmo no sólo depende del tamaño de la entrada, sino también de su forma. En estos casos, en el análisis del tiempo de ejecución de un algoritmo puede distinguirse entre:

- análisis de peor caso. $C_{worst}(n)$: máximo entre las entradas de tamaño n
- análisis de mejor caso. $C_{best}(n)$: mínimo entre las entradas de tamaño n
- análisis de caso promedio. $C_{avg}(n)$: promedio entre las entradas de tamaño n
 - el número de veces que la operación de interés será ejecutada para una entrada “típica”
 - NO es el promedio entre el mejor caso y el peor caso
 - el número esperado de operaciones de interés es interpretado como una variable aleatoria (con alguna suposición de la distribución de las posibles entradas)

Peor caso, caso promedio y mejor caso

En muchos casos, la eficiencia de un algoritmo no sólo depende del tamaño de la entrada, sino también de su forma. En estos casos, en el análisis del tiempo de ejecución de un algoritmo puede distinguirse entre:

- análisis de peor caso. $C_{worst}(n)$: máximo entre las entradas de tamaño n
- análisis de mejor caso. $C_{best}(n)$: mínimo entre las entradas de tamaño n
- análisis de caso promedio. $C_{avg}(n)$: promedio entre las entradas de tamaño n
 - el número de veces que la operación de interés será ejecutada para una entrada “típica”
 - NO es el promedio entre el mejor caso y el peor caso
 - el número esperado de operaciones de interés es interpretado como una variable aleatoria (con alguna suposición de la distribución de las posibles entradas)

Orden de Crecimiento

Orden de Crecimiento

El orden de crecimiento de la función asociada al tiempo de ejecución, entendida como la clase de funciones a la cual pertenece (o por la cual está acotada) la función de eficiencia de un algoritmo, es de gran importancia.

Orden de Crecimiento

El orden de crecimiento de la función asociada al tiempo de ejecución, entendida como la clase de funciones a la cual pertenece (o por la cual está acotada) la función de eficiencia de un algoritmo, es de gran importancia.

Esta tasa de crecimiento nos permite predecir respuestas a algunas preguntas importantes, del estilo de las siguientes:

Orden de Crecimiento

El orden de crecimiento de la función asociada al tiempo de ejecución, entendida como la clase de funciones a la cual pertenece (o por la cual está acotada) la función de eficiencia de un algoritmo, es de gran importancia.

Esta tasa de crecimiento nos permite predecir respuestas a algunas preguntas importantes, del estilo de las siguientes:

Orden de Crecimiento

El orden de crecimiento de la función asociada al tiempo de ejecución, entendida como la clase de funciones a la cual pertenece (o por la cual está acotada) la función de eficiencia de un algoritmo, es de gran importancia.

Esta tasa de crecimiento nos permite predecir respuestas a algunas preguntas importantes, del estilo de las siguientes:

¿Cuánto más rápido correrá este algoritmo en una computadora el doble de rápida de la que tengo?

Orden de Crecimiento

El orden de crecimiento de la función asociada al tiempo de ejecución, entendida como la clase de funciones a la cual pertenece (o por la cual está acotada) la función de eficiencia de un algoritmo, es de gran importancia.

Esta tasa de crecimiento nos permite predecir respuestas a algunas preguntas importantes, del estilo de las siguientes:

- ¿Cuánto más rápido correrá este algoritmo en una computadora el doble de rápida de la que tengo?
- ¿Cuánto tiempo más me insumirá resolver un problema del doble del tamaño?

Orden de Crecimiento

Orden de Crecimiento

Orden de Crecimiento

¿Cuánto más rápido correrá este algoritmo en una computadora el doble de rápida de la que tengo?

Orden de Crecimiento

¿Cuánto más rápido correrá este algoritmo en una computadora el doble de rápida de la que tengo?

Orden de Crecimiento

¿Cuánto más rápido correrá este algoritmo en una computadora el doble de rápida de la que tengo?

Si antes tardaba 10 minutos, ahora tardará **5 minutos**.

Orden de Crecimiento

¿Cuánto más rápido correrá este algoritmo en una computadora el doble de rápida de la que tengo?

Si antes tardaba 10 minutos, ahora tardará **5 minutos**.

Si antes tardaba 76 años, ahora tardará 38

Orden de Crecimiento

¿Cuánto más rápido correrá este algoritmo en una computadora el doble de rápida de la que tengo?

Si antes tardaba 10 minutos, ahora tardará **5 minutos**.

Si antes tardaba 76 años, ahora tardará 38

Orden de Crecimiento

¿Cuánto más rápido correrá este algoritmo en una computadora el doble de rápida de la que tengo?

Si antes tardaba 10 minutos, ahora tardará **5 minutos**.

Si antes tardaba 76 años, ahora tardará 38

¿Por qué entonces la tasa de crecimiento es importante para esta pregunta?

Orden de Crecimiento

¿Cuánto más rápido correrá este algoritmo en una computadora el doble de rápida de la que tengo?

Si antes tardaba 10 minutos, ahora tardará **5 minutos**.

Si antes tardaba 76 años, ahora tardará 38

¿Por qué entonces la tasa de crecimiento es importante para esta pregunta?

Orden de Crecimiento

¿Cuánto más rápido correrá este algoritmo en una computadora el doble de rápida de la que tengo?

Si antes tardaba 10 minutos, ahora tardará **5 minutos**.

Si antes tardaba 76 años, ahora tardará 38

¿Por qué entonces la tasa de crecimiento es importante para esta pregunta?

En un algoritmo eficiente $O(n)$: Si tu problema tarda 10 minutos y compras la máquina nueva, baja a 5 minutos. El ahorro de 5 minutos es bueno.

Orden de Crecimiento

¿Cuánto más rápido correrá este algoritmo en una computadora el doble de rápida de la que tengo?

Si antes tardaba 10 minutos, ahora tardará **5 minutos**.

Si antes tardaba 76 años, ahora tardará 38

¿Por qué entonces la tasa de crecimiento es importante para esta pregunta?

En un algoritmo eficiente $O(n)$: Si tu problema tarda 10 minutos y compras la máquina nueva, baja a 5 minutos. El ahorro de 5 minutos es bueno.

En tu algoritmo factorial $(n!)$: Si el problema tarda 76 años y compras la máquina nueva, ahora tarda 38 años. **El algoritmo sigue siendo igual de inutilizable.**

Orden de Crecimiento

Orden de Crecimiento

Asumiendo $C(n) = \frac{1}{2}n^2$, ¿Cuánto tiempo más me insumirá resolver un problema del doble del tamaño?

Orden de Crecimiento

Asumiendo $C(n) = \frac{1}{2}n^2$, ¿Cuánto tiempo más me insumirá resolver un problema del doble del tamaño?

$$\frac{T(2n)}{T(n)} = \frac{C_{op}C(2n)}{C_{op}C(n)} = \frac{\frac{1}{2}(2n)^2}{\frac{1}{2}n^2} = \frac{\frac{1}{2}4n^2}{\frac{1}{2}n^2} = 4$$

Orden de Crecimiento

Asumiendo $C(n) = \frac{1}{2}n^2$, ¿Cuánto tiempo más me insumirá resolver un problema del doble del tamaño?

$$\frac{T(2n)}{T(n)} = \frac{C_{op}C(2n)}{C_{op}C(n)} = \frac{\frac{1}{2}(2n)^2}{\frac{1}{2}n^2} = \frac{\frac{1}{2}4n^2}{\frac{1}{2}n^2} = 4$$

Orden de Crecimiento

Asumiendo $C(n) = \frac{1}{2}n^2$, ¿Cuánto tiempo más me insumirá resolver un problema del doble del tamaño?

$$\frac{T(2n)}{T(n)} = \frac{C_{op}C(2n)}{C_{op}C(n)} = \frac{\frac{1}{2}(2n)^2}{\frac{1}{2}n^2} = \frac{\frac{1}{2}4n^2}{\frac{1}{2}n^2} = 4$$

Insumirá 4 veces más!

Orden de Crecimiento

número de veces que la operación
de interés es ejecutada

Asumiendo $C(n) = \frac{1}{2}n^2$, ¿Cuánto tiempo más me insumirá resolver un problema del doble del tamaño?

$$\frac{T(2n)}{T(n)} = \frac{C_{op}C(2n)}{C_{op}C(n)} = \frac{\frac{1}{2}(2n)^2}{\frac{1}{2}n^2} = \frac{\frac{1}{2}4n^2}{\frac{1}{2}n^2} = 4$$

Insumirá 4 veces más!

¿Qué es un Polinomio?

Definición Formal: Un polinomio en n es una expresión de la forma:

$$P(n) = a_k n^k + a_{k-1} n^{k-1} + \dots + a_1 n + a_0$$

Coeficientes (a_i): Son números reales **constantes**. El coeficiente principal (a_k) debe ser distinto de cero.

Exponentes (k): Deben ser **números enteros no negativos** ($k \in \mathbb{N}_0$).

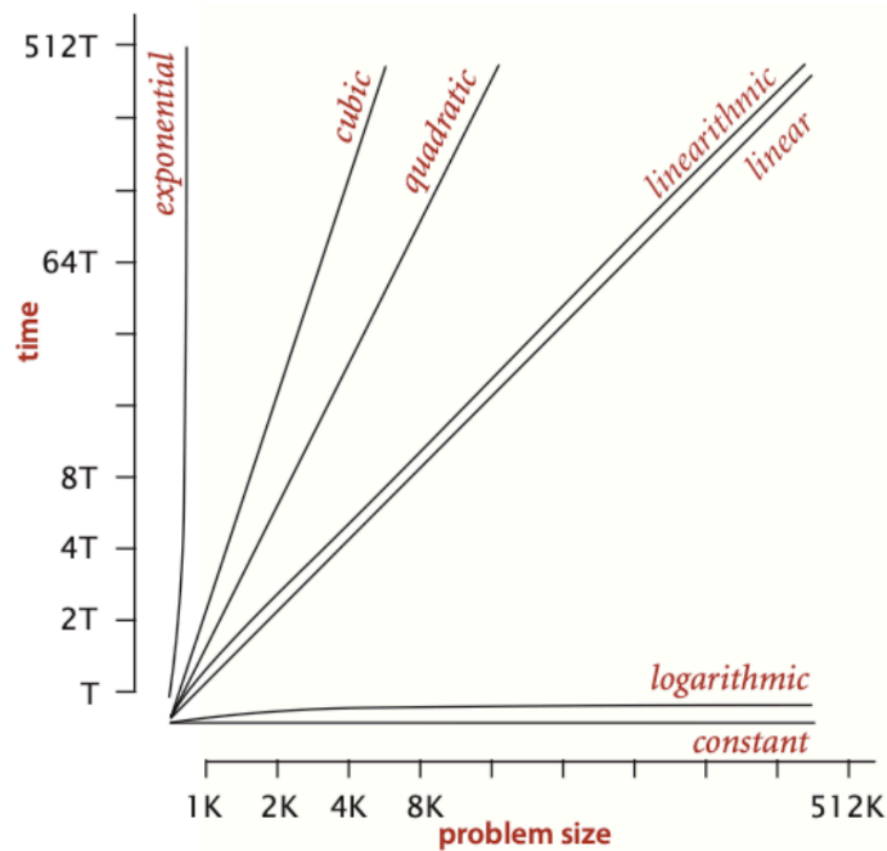
¿Qué es un Polinomio?

$f(n) = n^{1/2} = \sqrt{n}$ El exponente no es entero.

$f(n) = \log n$ Es una función, no una constante

$f(n) = 2^n$ Es una función exponencial (la variable está en el exponente, no en la base).

Algunas ordenes de crecimiento importantes



Algunas ordenes de crecimiento importantes

Algunas ordenes de crecimiento importantes

ent.	$\log_2 n$	n	$n \log_2 n$	n^2	n^3	2^n	$n!$
10	3.3	10^1	3.3×10^1	10^2	10^3	10^3	3.6×10^6
10^2	6.6	10^2	6.6×10^2	10^4	10^6	1.3×10^{30}	9.3×10^{157}
10^3	10	10^3	10^4	10^6	10^9		
10^4	13	10^4	1.3×10^5	10^8	10^{12}		
10^5	17	10^5	1.7×10^6	10^{10}	10^{15}		
10^6	20	10^6	2×10^7	10^{12}	10^{18}		

Asintóticas de Tasas de Crecimiento

Una forma útil de comparar funciones de eficiencia ignorando factores constantes (y tamaños pequeños en las entradas) es a través de la consideración de asintóticas de la tasa de crecimiento de las funciones de eficiencia.

La notación usual para clases de funciones de eficiencia, relacionada a asintóticas, es la siguiente:

$O(g(n))$: Clase de funciones que no crecen más rápido que g

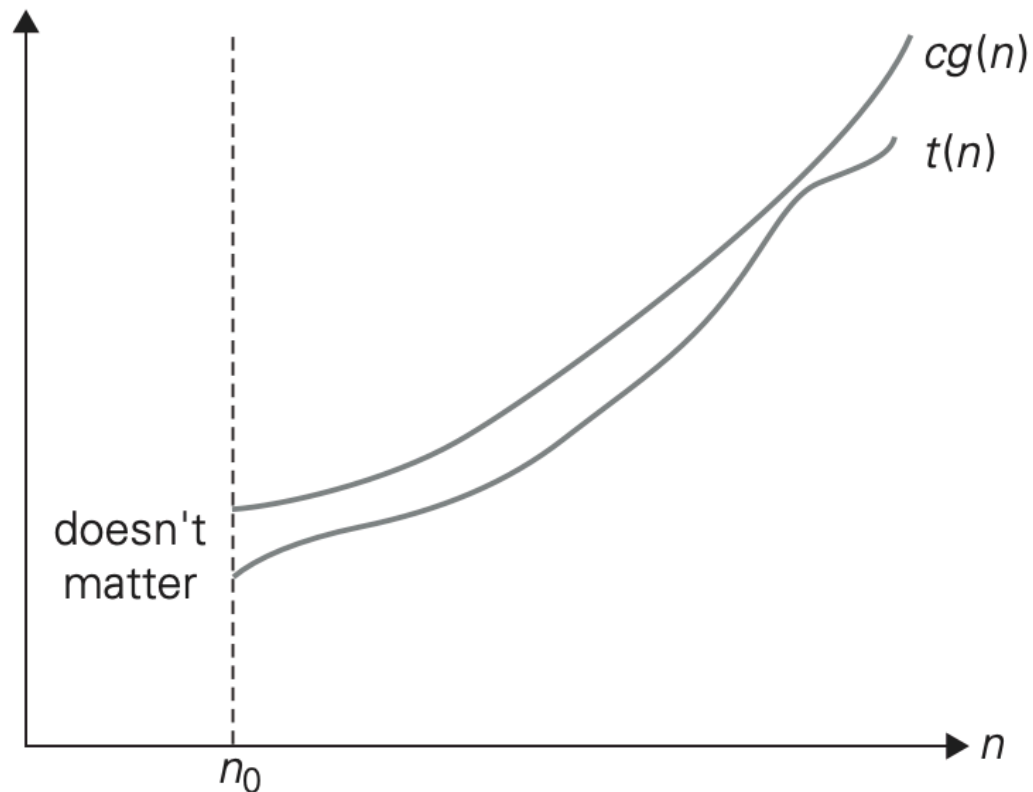
$\Omega(g(n))$: Clase de funciones que no crecen menos rápido que g

$\Theta(g(n))$: Clase de funciones que tienen exactamente la misma tasa de crecimiento que g

Asintóticas de Tasas de Crecimiento

$O(g(n))$

$$\exists n_0 \geq 0, c > 0 \mid \forall n \geq n_0 : t(n) \leq cg(n)$$



Ejemplo

$$100n + 5 \in O(n^2)$$

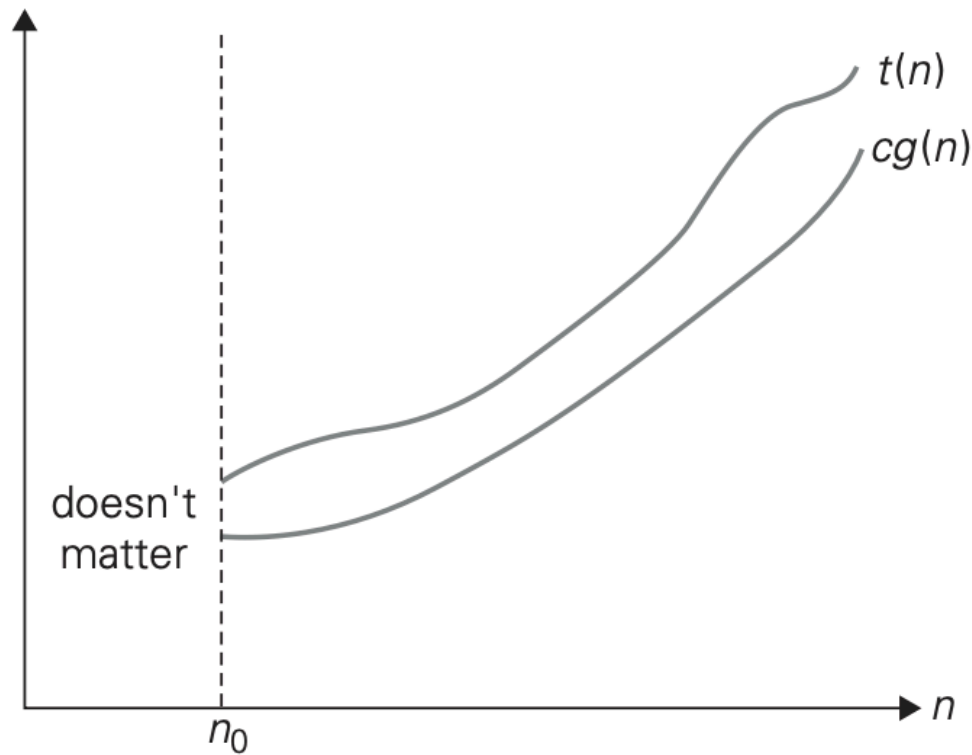
$$100n + 5 \leq 100n + n \text{ (for all } n \geq 5) = 101n \leq 101n^2. \quad n_0 = 5, c = 101$$

La definición nos da mucha libertad para elegir valores específicos para las constantes c y n_0 , por ejemplo:

$$100n + 5 \leq 100n + 5n \text{ (for all } n \geq 1) = 105n \quad n_0 = 1, c = 105$$

Asintóticas de Tasas de Crecimiento

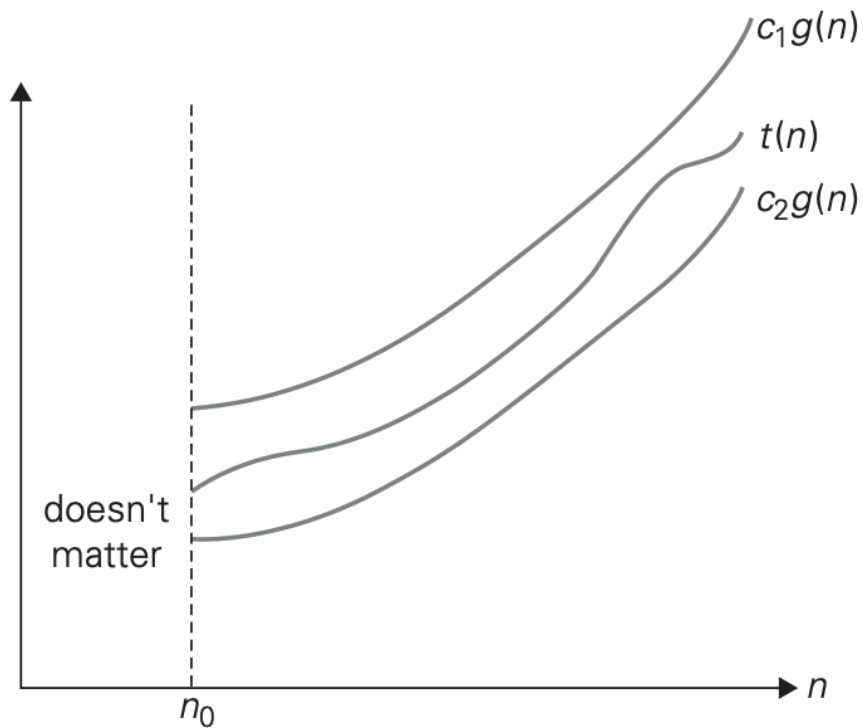
$$\Omega(g(n)) \quad \exists n_0 \geq 0, c > 0 \mid \forall n \geq n_0 : t(n) \geq cg(n)$$



Asintóticas de Tasas de Crecimiento

$$\exists n_0 \geq 0, c_1 > 0, c_2 > 0 \mid \forall n \geq n_0 : c_2 g(n) \leq t(n) \leq c_1 g(n)$$

$\Theta(g(n))$



Ejemplos

$$n \in O(n^2)$$

$$n^3 \notin O(n^2)$$

$$\frac{1}{2}n(n-1) \in O(n^2)$$

$$n^3 \in \Omega(n^2)$$

$$n^2 + bc + c \in \Theta(n^2)$$

Clases Básicas de Eficiencia

1	constante
$\log n$	logarítmica
n	lineal
$n \log n$	n-log-n
n^2	cuadrática
n^3	cúbica
2^n	exponencial
$n!$	factorial

Algoritmos que requieren un número exponencial de operaciones son prácticos solo para resolver instancias muy pequeñas del problema

Clases Básicas de Eficiencia

1	constante
$\log n$	logarítmica
n	lineal
$n \log n$	n-log-n
n^2	cuadrática
n^3	cúbica
2^n	exponencial
$n!$	factorial

Algoritmos que requieren un número exponencial de operaciones son prácticos solo para resolver instancias muy pequeñas del problema

Análisis de Eficiencia de Algoritmos Iterativos

Análisis de Eficiencia de Algoritmos Iterativos

Las actividades usuales asociadas al análisis de eficiencia de algoritmos iterativos son las siguientes:

Análisis de Eficiencia de Algoritmos Iterativos

Las actividades usuales asociadas al análisis de eficiencia de algoritmos iterativos son las siguientes:

- Decidir **cuál es el tamaño de la entrada** para el problema en cuestión

Análisis de Eficiencia de Algoritmos Iterativos

Las actividades usuales asociadas al análisis de eficiencia de algoritmos iterativos son las siguientes:

- Decidir **cuál es el tamaño de la entrada** para el problema en cuestión
- **Identificar la operación de interés** para el algoritmo

Análisis de Eficiencia de Algoritmos Iterativos

Las actividades usuales asociadas al análisis de eficiencia de algoritmos iterativos son las siguientes:

- Decidir **cuál es el tamaño de la entrada** para el problema en cuestión
- **Identificar la operación de interés** para el algoritmo
- Determinar **cuáles son los casos** que corresponden a las configuraciones **peor**, **mejor** o **promedio** para el algoritmo en cuestión (de acuerdo al análisis que se quiera realizar)

Análisis de Eficiencia de Algoritmos Iterativos

Las actividades usuales asociadas al análisis de eficiencia de algoritmos iterativos son las siguientes:

- Decidir **cuál es el tamaño de la entrada** para el problema en cuestión
- **Identificar la operación de interés** para el algoritmo
- Determinar **cuáles son los casos** que corresponden a las configuraciones **peor**, **mejor** o **promedio** para el algoritmo en cuestión (de acuerdo al análisis que se quiera realizar)
- **Obtener una expresión** que denote el **número de veces que la operación de interés es ejecutada** (para cada uno de los casos anteriores)

Análisis de Eficiencia de Algoritmos Iterativos

Las actividades usuales asociadas al análisis de eficiencia de algoritmos iterativos son las siguientes:

- Decidir **cuál es el tamaño de la entrada** para el problema en cuestión
- **Identificar la operación de interés** para el algoritmo
- Determinar **cuáles son los casos** que corresponden a las configuraciones **peor**, **mejor** o **promedio** para el algoritmo en cuestión (de acuerdo al análisis que se quiera realizar)
- **Obtener una expresión** que denote el **número de veces que la operación de interés es ejecutada** (para cada uno de los casos anteriores)
- **Simplificar la expresión** anterior (que suele involucrar sumatorias)

Ejemplo 1: Búsqueda Secuencial

Realicemos para este ejemplo el análisis de tiempo de ejecución:

Algoritmo *int BusquedaSecuencial(int A[0..n-1], int K)*

*// Busca un valor dado en un arreglo, mediante búsqueda
secuencial.*

// Entrada: un arreglo A[0..n-1] y una clave K

// Salida: el índice de la primera posición de A en que está

// almacenado K, o -1 si K no pertenece a A

i ← 0

while *i < n and A[i] ≠ K do*

i ← i+1

if *i < n then return i*

else return -1

Análisis de tiempo de Ejecución

Búsqueda Secuencial

Operación de interés: comparación de claves

Tamaño de las entradas: Tamaño de la lista

- Worst-Case: El elemento no esta en la lista o esta en el último lugar.

$$C_{worst}(n) = \sum_{j=0}^{n-1} 1 = n \quad \theta(n)$$

- Best-Case: Elemento buscado se encuentra en la primera posición de la lista

$$C_{best}(n) = 1 \quad \theta(1)$$

Ejemplo 2: Selection Sort

Realicemos para este segundo ejemplo el análisis de tiempo de ejecución:

Algoritmo SelectionSort($A[0..n-1]$)

// Ordena un arreglo mediante el proceso de selección.

// Entrada: un arreglo $A[0..n-1]$ de elementos comparables

// Salida: el arreglo $A[0..n-1]$, ordenado de manera ascendente.

for $i \leftarrow 0$ **to** $n-2$ **do**

$min \leftarrow i$

for $j \leftarrow i+1$ **to** $n-1$ **do**

if $A[j] < A[min]$ **then**

$min \leftarrow j$

swap($A[i], A[min]$)

Análisis de tiempo de Ejecución

Selection Sort

Operación de interés: comparación de claves

Todas las entradas de tamaño producen el mismo tiempo de ejecución

Algoritmo SelectionSort($A[0..n-1]$)

// Ordena un arreglo mediante el proceso de selección.

// Entrada: un arreglo $A[0..n-1]$ de elementos comparables

// Salida: el arreglo $A[0..n-1]$, ordenado de manera ascendente.

for $i \leftarrow 0$ **to** $n-2$ **do**

$min \leftarrow i$

for $j \leftarrow i+1$ **to** $n-1$ **do**

if $A[j] < A[min]$ **then**

$min \leftarrow j$

swap($A[i], A[min]$)

$$C(n) = \sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} 1 = \sum_{i=0}^{n-2} [(n-1) - (i+1) + 1] = \sum_{i=0}^{n-2} (n-1-i).$$
$$= \frac{(n-1)n}{2}.$$

Análisis de tiempo de Ejecución Selection Sort

$$\sum_{i=0}^{n-2} (n-1-i) =$$

$$(n-1) + (n-2) + (n-3) + \dots + (n-1-(n-2))$$
$$(n-1) + (n-2) + (n-3) + \dots + \underbrace{n-1-n+2}_1$$

$$1 + 2 + \dots + (n-3) + (n-2) + (n-1)$$

$$= \sum_{i=1}^{n-1} i = \frac{(n-1) \cdot (n-1+1)}{2}$$

$$= \frac{(n-1) \cdot n}{2}$$

Análisis de Eficiencia de Algoritmos Recursivos

Análisis de Eficiencia de Algoritmos Recursivos

Las actividades usuales asociadas al análisis de eficiencia de algoritmos recursivos son las siguientes:

Análisis de Eficiencia de Algoritmos Recursivos

Las actividades usuales asociadas al análisis de eficiencia de algoritmos recursivos son las siguientes:

- Decidir **cuál es el tamaño de la entrada** para el problema en cuestión

Análisis de Eficiencia de Algoritmos Recursivos

Las actividades usuales asociadas al análisis de eficiencia de algoritmos recursivos son las siguientes:

- Decidir **cuál es el tamaño de la entrada** para el problema en cuestión
- **Identificar la operación de interés** para el algoritmo

Análisis de Eficiencia de Algoritmos Recursivos

Las actividades usuales asociadas al análisis de eficiencia de algoritmos recursivos son las siguientes:

- Decidir **cuál es el tamaño de la entrada** para el problema en cuestión
- **Identificar la operación de interés** para el algoritmo
- **Determinar** cuáles son los **casos** que corresponden a las configuraciones **peor**, **mejor** o **promedio** para el algoritmo en cuestión (de acuerdo al análisis a realizar)

Análisis de Eficiencia de Algoritmos Recursivos

Las actividades usuales asociadas al análisis de eficiencia de algoritmos recursivos son las siguientes:

- Decidir **cuál es el tamaño de la entrada** para el problema en cuestión
- **Identificar la operación de interés** para el algoritmo
- **Determinar** cuáles son los **casos** que corresponden a las configuraciones **peor**, **mejor** o **promedio** para el algoritmo en cuestión (de acuerdo al análisis a realizar)
- **Obtener una ecuación de recurrencia** que denote **el número de veces que la operación de interés es ejecutada** (para cada uno de los casos anteriores)

Análisis de Eficiencia de Algoritmos Recursivos

Las actividades usuales asociadas al análisis de eficiencia de algoritmos recursivos son las siguientes:

- Decidir **cuál es el tamaño de la entrada** para el problema en cuestión
- **Identificar la operación de interés** para el algoritmo
- **Determinar** cuáles son los **casos** que corresponden a las configuraciones **peor**, **mejor** o **promedio** para el algoritmo en cuestión (de acuerdo al análisis a realizar)
- **Obtener una ecuación de recurrencia** que denote **el número de veces que la operación de interés es ejecutada** (para cada uno de los casos anteriores)
- **Resolver la expresión** anterior hasta obtener una forma “cerrada”

Análisis de tiempo de Ejecución $n!$

$$0! = 1$$

$$n! = 1.2. \dots .(n-1).n = (n-1)! \cdot n \text{ for } n \geq 1$$

```
public static long factorial(int n) {  
    if (n == 0) {  
        return 1;  
    } else {  
        return factorial(n - 1) * n;  
    }  
}
```

Operación de interés

Tamaño de la entrada: n en si mismo

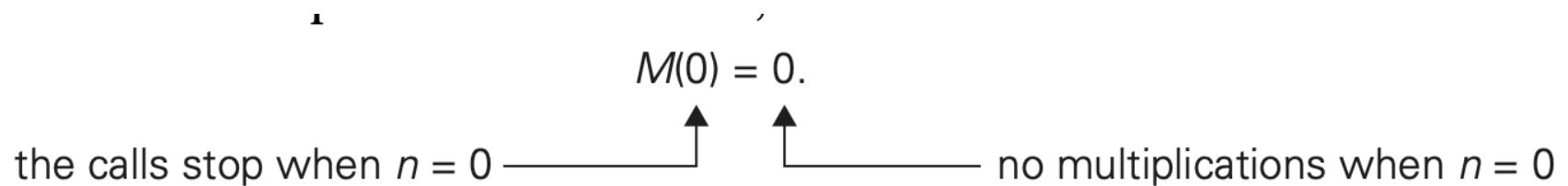
Análisis de tiempo de Ejecución n!

$$M(n) = M(n-1) + 1 \quad \text{for } n > 0.$$

to compute
to multiply

$F(n-1)$
 $F(n-1)$ by n

Cuando $n=0$ el algoritmo realiza cero multiplicaciones



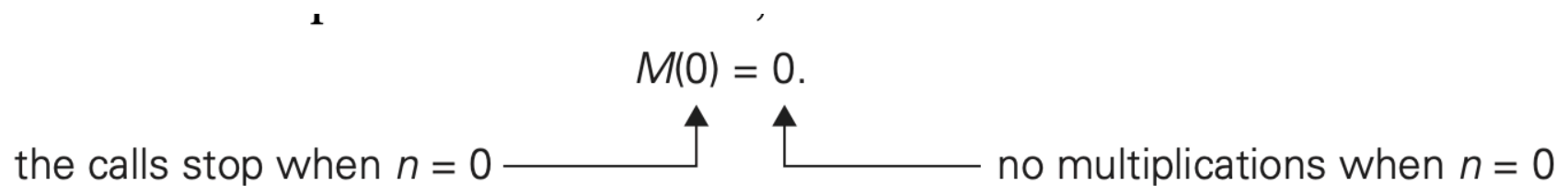
Análisis de tiempo de Ejecución $n!$

$M(n)$: cantidad de multiplicaciones realizadas

$$M(n) = M(n-1) + 1 \quad \text{for } n > 0.$$

to compute $F(n-1)$ to multiply $F(n-1)$ by n

Cuando $n=0$ el algoritmo realiza cero multiplicaciones



Análisis de tiempo de Ejecución: $n!$

$$\begin{aligned} M(n) &= M(n - 1) + 1 \quad \text{for } n > 0, \\ M(0) &= 0. \end{aligned}$$

Análisis de tiempo de Ejecución: $n!$

$$M(n) = M(n - 1) + 1 \quad \text{for } n > 0, \\ M(0) = 0.$$

$$M(n) = M(n-1) + 1$$

$$i = 1$$

$$= (M(n-1-1) + 1) + 1$$

$$= M(n-2) + 2$$

$$i = 2$$

$$= (M(n-2-1) + 1) + 2$$

$$= M(n-3) + 3$$

$$i = 3$$

$$= M(n-i) + i$$

Análisis de tiempo de Ejecución: $n!$

$$M(n) = M(n - 1) + 1 \quad \text{for } n > 0,$$
$$M(0) = 0.$$

$$M(n) = M(n-1) + 1 \quad i=1$$

$$= (M(n-1-1) + 1) + 1$$

$$= M(n-2) + 2 \quad i=2$$

$$= (M(n-2-1) + 1) + 2$$

$$= M(n-3) + 3 \quad i=3$$

$$= M(n-i) + i$$

$$n-i=0$$
$$n=i$$

$$M(n-n) + n =$$

$$M(0) + n =$$

$$0 + n = n$$

Análisis de tiempo de Ejecución: $n!$

$$M(n) = M(n - 1) + 1 \quad \text{for } n > 0,$$
$$M(0) = 0.$$

$$M(n) = M(n-1) + 1 \quad i=1$$

$$= (M(n-1-1) + 1) + 1$$

$$= M(n-2) + 2 \quad i=2$$

$$= (M(n-2-1) + 1) + 2$$

$$= M(n-3) + 3 \quad i=3$$

$$= M(n-i) + i$$

$$n-i=0$$
$$n=i$$

$$M(n-n) + n =$$

$$M(0) + n =$$

$$0 + n = n$$

Análisis de tiempo de Ejecución

Otro ejemplo

```
/Input: A positive decimal integer n
//Output: The number of binary digits in n's binary representation
Public static int BinRec(int n){
    if n = 1
        return 1
    else
        return BinRec([n/2]) + 1
}
```

Operación de interés

Tamaño de la entrada: n en si mismo

Número de sumas para computar $\text{BinRec}(\lfloor n/2 \rfloor)$

$$A(n) = A\left(\frac{n}{2}\right) + 1$$

$$A(1) = 0$$

Análisis de tiempo de Ejecución

Otro ejemplo

$$A(1) = 0,$$

$$A(n) = A\left(\frac{n}{2}\right) + 1 = (A\left(\frac{n}{4}\right) + 1) + 1 = A\left(\frac{n}{4}\right) + 2$$

$$= (A\left(\frac{n}{8}\right) + 1) + 2 = A\left(\frac{n}{8}\right) + 3$$

$$= (A\left(\frac{n}{2^i}\right) + i), \text{ sea } i = \log_2 n, \quad = A\left(\frac{n}{2^{\log_2 n}}\right) + \log_2 n = A\left(\frac{n}{n^{\log_2 2}}\right) + \log_2 n$$

$$b^{\log_b n} = n^{\log_b b} = n$$

$$= A\left(\frac{n}{n^{\log_2 2}}\right) + \log_2 n = A(1) + \log_2 n = \log_2 n$$

$$A(n) = \log_2 n \in \Theta(\log n).$$

Análisis de tiempo de Ejecución

La presencia de $\lfloor n/2 \rfloor$ complica el uso directo de sustituciones si n no es potencia de 2.

En estos casos, podemos aplicar el siguiente enfoque:

1. Resolver la recurrencia para $n = 2^k$.
2. Aplicar la **regla de suavidad**: el orden de crecimiento para potencias de 2 se mantiene para todo n .

Análisis de Tiempo de Ejecución

Asumimos que n es par

```
int count = 0;
for (int i = 0; i < n; i++) {
    if (i % 2 == 0) {
        for (int j = 0; j < n; j++)
            count++; // <-- Operación de Interés
    }
    else {
        for (int j = 0; j < 10; j++)
            count++; // <-- Operación de interés
    }
}
```

Análisis de Tiempo de Ejecución

```
int count = 0;
for (int i = n/2; i < n; i++)
    for (int j = 1; j < n; j = 2 * j)
        for (int k = 1; k < n; k = 2 * k)
            count++; // <-- Operación primitiva de interés
```

Análisis de Tiempo de Ejecución

```
Function subSets(conjunto) {  
    powerSet = { {} } // Empezamos con el conjunto vacío adentro  
    for (valor : conjunto) {  
        new powerSetAux = { } // Conjunto vacío temporal  
        for (t : powerSet) {  
            new nuevoSubSet = t u {valor}  
            powerSetAux = powerSetAux u {nuevoSubSet}  
        }  
  
        powerSet = powerSet u powerSetAux  
    }  
  
    return powerSet  
}
```


Análisis Empírico de Eficiencia en Tiempo

En muchos casos, **realizar una evaluación analítica** del tiempo de ejecución de un algoritmo **puede ser demasiado complicado**. En estos casos, una **alternativa** útil es realizar un **análisis empírico**. Algunos de los **puntos a tener en cuenta** en el análisis empírico de eficiencia son los siguientes:

- seleccionar una muestra específica (típica) de entradas para el algoritmo
- utilizar unidades de tiempo para la medida, o contar el número de operaciones de interés realizadas
- analizar los datos obtenidos

El diseño de un análisis empírico debe evitar cuidadosamente diferentes tipos de **amenazas de validez**